Conn2Conn: the three grid estimators

PCA–PLS, PCA–BayesianRidge, PCA–KernelRidge(RBF)

Reproduction grid (`reproduction/_grid_common.py`)

The reproducibility grid evaluates exactly **three estimators**, expanded into **11 estimator-variants** per cell (`pca_pls`×1 + `bayesian_ridge`×1 + `kernel_ridge`×9). All three share one scaffold and differ only in the regressor that lives in the compressed latent space. This note describes that shared scaffold first (§1), then each estimator (§2–§4), then the two structural variants every estimator is run through (§5).

1 The shared scaffold (the parts common to all three)

Every connectome is an **upper-triangle edge vector** of a region×region matrix (Glasser: 64,620 edges; 4S456Parcels: 103,740). Predicting one connectome from another is a regression from a $\sim 10^5$-dim input to a $\sim 10^5$-dim output with only $n_{\text{train}} \approx 683$ subjects — hopelessly underdetermined in edge space. Every estimator therefore runs the same five-part pipeline; only Part 3 changes.

P1. Source compression. Fit PCA on the training input and project both partitions:

$$Z^{\text{src}} = \text{PCA}_{k_{\text{src}}}(X_{\text{tr}}) \cdot X, \qquad k_{\text{src}} = \min(256,\ \text{width}).$$

P2. Target compression. Fit a *separate* PCA on the training target:

$$Z^{\text{tgt}}_{\text{tr}} = \text{PCA}_{k_{\text{tgt}}}(Y_{\text{tr}}) \cdot Y_{\text{tr}}, \qquad k_{\text{tgt}} = \min(256,\ \text{width}).$$

P3. Latent regressor. Learn $f : Z^{\text{src}} \to Z^{\text{tgt}}$ on the train latents. *This is the only part that differs between the three estimators.*

P4. Inverse-PCA. Map the predicted target latents back to edge space: $\hat{Y} = \text{PCA}^{-1}_{k_{\text{tgt}}}\big(f(Z^{\text{src}}_{\text{te}})\big)$.

P5. Evaluation. `full_panel_eval` computes the panel {`mse`, `r2`, `pearson`, `demeaned_pearson`, `top1_acc`, `avg_rank`}. The load-bearing metric is `demeaned_pearson`: the per-subject cosine between predicted and true connectomes *after subtracting the training group-mean* — i.e. how well individual deviation from the average brain is recovered (raw pearson is ≈ 0.8–0.9 for everything and meaningless).

The low-dim cap ($k_{\text{src}} = 256$, $k_{\text{tgt}} = 256$, $k_{\text{pls}} = 64$). Baked into `_grid_common.capped_*`. It is the central regularizer: it forces every model to operate in a $\leq$ 256-dim subspace, which is why "bigger model / more data" does not help (F10) — the bottleneck is the rank of the shared FC↔SC subspace, not model capacity. Narrow inputs (`bv` ∼16-dim, `demo`) are capped to their own width.

2 Estimator 1 — `pca_pls` (PCA → PLS → inverse-PCA)

Role: the closed-form workhorse for *reconstruction* headlines. **Part 3:** a single Partial-Least-Squares regression jointly mapping all source latents to all target latents,

$$\text{PLSRegression}\big(n_{\text{components}} = k_{\text{pls}} = 64,\ \texttt{scale=True},\ \texttt{max\_iter} = 2000\big).$$

PLS finds paired projections of Z^{src} and Z^{tgt} with maximal covariance, so it captures *shared* cross-modal directions rather than fitting each target dimension independently. `scale=True` standardizes columns internally — this is what protected the reconstruction path from the float32 ill-conditioning that bit the scalar path (§5). Closed-form, deterministic, fast. Implementation: `pca_pls_predict` in `_setup.py`.

3 Estimator 2 — `bayesian_ridge` (PCA → per-component Bayesian-Ridge)

Role: the strongest oracle and **the only stable estimator for scalar / cognition targets**. **Part 3:** fit one independent `BayesianRidge` per target PCA component,

$$\hat{z}_k^{\text{tgt}} = \text{BayesianRidge}(\texttt{max\_iter}{=}300) \text{ fit on } (Z_{\text{tr}}^{\text{src}},\ z_{\text{tr},k}^{\text{tgt}}), \qquad k = 1, \ldots, k_{\text{tgt}}.$$

Bayesian ridge places a Gaussian prior on the weights and *infers* the ridge penalty (precision α, λ) from the data per component via evidence maximization — so each of the ≤ 256 target modes gets its own automatically-tuned regularization, no CV needed. Slower than PLS (k_{tgt} separate fits) but the per-mode adaptivity makes it the most reliable estimator for ill-conditioned targets. Implementation: `br_per_component_predict`.

Reporting rule: reconstruction headlines may use either `pca_pls` or `bayesian_ridge`; **downstream/cognition must use** `bayesian_ridge` — the PDF's oracle (0.647) and cognition baseline (0.359) reproduce to the digit only when the estimator is held to BR (the "numbers don't match" scare was estimator confusion).

4 Estimator 3 — `kernel_ridge` (PCA → standardize → KRR-RBF), a 3×3 sweep

Role: the trustworthy *nonlinear probe* — the null that says "you can't recover the FC↔SC gap with a nonlinear model." **Part 3:** standardize the source latents (`StandardScaler`), then fit a multi-output kernel ridge regression with an RBF kernel mapping to the target latents,

$$\hat{Z}^{\text{tgt}} = \text{KernelRidge}\big(\texttt{kernel=rbf},\ \alpha,\ \gamma\big), \qquad \gamma = \gamma_{\text{med}} \cdot m_\gamma.$$

- **Bandwidth** γ is set by the *median pairwise-distance heuristic* $\gamma_{\text{med}} = 1/(2 \cdot \text{median}\,\|z_i - z_j\|^2)$ on a deterministic 300-point subsample (`_median_gamma`), then scaled by a multiplier $m_\gamma \in \{0.5, 1.0, 2.0\}$.
- **Ridge penalty** $\alpha \in \{0.1, 1.0, 10.0\}$.

The 3×3 grid of (m_γ, α) gives the **9 variants**, tagged `kernel_ridge[alpha=`a`,gamma_mult=`g`]`. **Finding:** the sweep is *degenerate* — `demeaned_pearson` spreads only ~ 0.004 across all 9 cells, so all 9 collapse to effectively one model. That flatness *is* the result: nonlinearity and HP tuning buy nothing in this regime. Implementation: `_kr_single` / `_kr_grid`; prototype `kernelridge_predict` in `_tract_setup.py`.

5 The two structural variants each estimator runs through

Each estimator above is exposed as both a *single-matrix* function and a *block* function, and is reused on a separate *scalar* path for downstream. These are the same three models, re-plumbed — not new models.

(a) Single vs. Block (BP-1: per-block scaling). For plain connectome inputs (`FC`, `SC`) the source PCA is fit on one matrix. For mixed inputs `FC+bv+demo` / `SC+bv+demo`, a naive concatenation would let the 10^5-dim connectome swamp the tiny ∼16-dim subject-info block in a single shared PCA. The **block** path (`capped_block_*`, `block_pca_pls_predict`) instead gives each block its own latent budget: wide blocks get `PCA`($k_{\text{per\_block}}$=256), narrow blocks (`bv`, `demo`) are z-scored and passed through raw, then the latents are concatenated before Part 3. This is **BP-1**; it is why `demo` can still influence a prediction dominated by a huge connectome.

(b) Scalar (downstream) path. For scalar targets — cognition (`CogTotal/Fluid/Cryst`) and the sex/age leak-checks — the target PCA (P2) and inverse-PCA (P4) are dropped; the regressor predicts the scalar directly (`pca_pls_scalar`, `bayesian_ridge_scalar`, `_kr_scalar`). Two differences matter:

- **float64 before PCA.** `bv+demo` contains exactly-collinear one-hot columns (`sex`/`race` each sum to 1 ⇒ rank-deficient). float32 SVD is ill-conditioned on that and *lost the demographic directions split-dependently* (e.g. `bv+demo`→`age` collapsed to 0.46 on some seeds; float64 → 1.000). The reconstruction path was shielded by `PLSRegression(scale=True)`; the scalar path was not, hence the explicit `_f64` cast. (This was the June leak-diagnostics bug fix.)
- For scalar PLS, `n_components` is capped at $\min(32, k)$ (one scalar target needs far fewer components than a 256-mode connectome).

Downstream is judged by `lift_over_bvdemo` (score minus the `bv+demo` baseline, same estimator/seed) with a paired sign-flip permutation p (`paired_permutation_lift_p`); sex/age are leak-checks, not findings.

Summary table

Estimator	Latent regressor (P3)	Why / role	Variants
`pca_pls`	PLS, k_{pls}=64, `scale=True`	closed-form workhorse; shared-covariance directions; reconstruction headline	1
`bayesian_ridge`	per-component BayesianRidge, evidence-tuned penalty	strongest oracle; *only* stable scalar estimator (cognition)	1
`kernel_ridge`	RBF KRR, median-γ, (m_γ, α) sweep	nonlinear null probe; degenerate ($\Delta \sim 0.004$)	9 (3×3)

Not in the grid (deliberately): Krakencoder, learnable PLS, MLP/GNN, CovProjector — those live in the notebook exploration; the grid is the linear/closed-form confirmatory spine plus one honest nonlinear null.